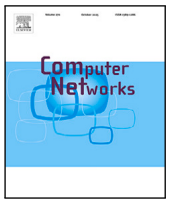
Computer Networks 283 (2026) 112319



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet



Cooperative path scheduling and resource allocation for LEO satellite-enabled mobile edge computing network

Cheng Gao, Bo Hu *, Meng Meng

State Key Lab of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, 100876, China

ARTICLE INFO

Keywords:
Low-earth orbit satellite
Mobile edge computing
Path scheduling
Resource allocation

ABSTRACT

Low Earth orbit (LEO) satellites equipped with computing capabilities can provide task offloading services for terrestrial users. Moreover, through inter-satellite collaboration, multiple LEO satellites can jointly process tasks and enhance service performance. However, the highly dynamic topology of LEO satellite networks and the uneven distribution of available computing resources across satellites pose significant challenges in task scheduling. Specifically, it remains critical to determine which satellite should execute a given task and how to allocate communication and computing resources, in order to ensure efficient and reliable service delivery. In this paper, we investigate the path scheduling and resource allocation in the satellites collaborative computing network, taking into account tasks splitting, offloading target selection, transmission path optimization, and allocation of communication and computing resources. We formulate a joint path scheduling and resource allocation optimization problem to optimize task completion delay and users' energy consumption, which is a Mixed-Integer Non-Linear Programming (MINLP) problem. Subsequently, we transform the problem into two subproblems, the total delay minimization subproblem and minimizing the energy consumption minimization subproblem. And then, we design penalty-based successive convex approximation (SCA) algorithms and alternating iteration algorithms to solve them. We develop an initialization algorithm based on load balancing and the Floyd shortest path method to obtain initial path scheduling strategies. Simulation results show that the proposed algorithms exhibit rapid convergence and outperform three benchmark algorithms in terms of performance.

1. Introduction

With the rapid advancement of satellite communication technology, low earth orbit (LEO) satellite communication has emerged as a key focus for 6G and future networks, attracting growing interest from researchers and companies worldwide [1]. Currently, there are several operational and in-progress LEO satellite constellations, including OneWeb, Iridium, and Starlink [2–4]. These constellations offer global coverage, enabling communication and data transmission services for users across the globe [5]. For example, the Starlink constellation comprises more than 8000 satellites, providing terrestrial users with upload speeds of 150–500 Mbps and latency as low as 20–40 ms [6]. Moreover, companies like Starlink, China Telecom, and Apple are actively researching direct satellite connectivity for smartphones [7]. Starlink has already introduced the Direct-to-Device feature on its latest set of six satellites [8,9]. Hence, delivering communication services directly to terrestrial users via LEO satellites represents a significant advancement in satellite network development. This paper assumes that all terrestrial users can engage in direct wireless communication with LEO satellites.

Deploying edge servers on low Earth orbit (LEO) satellites enables direct task processing for terrestrial users [10]. This approach eliminates the need to forward tasks to the terrestrial data network and processes them directly on the satellite, which can decrease users' tasks processing latency, and enhance the quality of experience (QoE) of users. It is considered an important development direction for satellite networks. In practical deployments, LEO satellite networks equipped with onboard edge computing capabilities are expected to support a wide range of mission-critical services. For example, through cooperation with terrestrial cellular networks, LEO satellites can provide complementary coverage for remote, oceanic, and sparsely populated regions where ground infrastructure is unavailable or economically infeasible, allowing computation-intensive tasks (e.g., sensing, surveillance, or navigation services) to be offloaded to satellites to ensure service continuity. In addition, during emergencies such as natural disasters, earthquakes, or large-scale power outages, terrestrial communication systems may be damaged or overloaded, whereas LEO

* Corresponding author.
E-mail addresses: gaoch@bupt.edu.cn (C. Gao), hubo@bupt.edu.cn (B. Hu), m_meng@bupt.edu.cn (M. Meng).

https://doi.org/10.1016/j.comnet.2026.112319
Received 17 April 2025; Received in revised form 2 January 2026; Accepted 14 April 2026
Available online 17 April 2026
1389-1286/© 2026 Published by Elsevier B.V.

satellites with edge computing capabilities can rapidly establish temporary service networks to support real-time data processing for disaster assessment and rescue coordination. Beyond these representative scenarios, emerging services such as large-scale environmental sensing, maritime and aviation communications, and intelligent transportation systems also demand reliable and low-latency computation support over highly dynamic satellite networks.

However, LEO satellites face inherent limitations in payload capacity, constraining the computing capabilities of their onboard edge servers [11,12]. Furthermore, user task demands exhibit substantial spatial variability due to differences in geographic location and terrestrial network coverage [13]. Consequently, some satellites experience computational resource shortages while others remain underutilized.

To optimize computing resource utilization on LEO satellites and enhance user service capabilities, inter-satellite links (ISLs) can facilitate task forwarding across satellites, enabling distributed processing. This architecture constitutes a LEO satellite-enhanced mobile edge computing network [14], as shown in Fig. 1. In this process, further research is needed on how to schedule tasks (including which satellite the task is offloaded to, and which ISLs the task is forwarded through), and how communication and computing resources are allocated.

Based on above discussion, we investigate the path scheduling and resource allocation for LEO Satellite-enhanced mobile edge computing network. In this network, each user's task can be split to two parts. One part is processed on the user's local CPU, and the other part is offloaded to the satellite and processed on the edge server of the LEO satellite. During this process, we have optimized the allocation of communication and computing resources. The main contributions of this paper can be summarized as follows:

- We propose a LEO satellite-enhanced mobile edge computing network, where LEO satellites collaborate to provide edge computing services to terrestrial users through inter-satellite links. To minimize the energy consumption of all users and the total completion delay of all tasks, we formulate a optimization problem where task offloading nodes selection, task scheduling paths planning, communication and computing resources allocation are jointly optimized. This problem is a mixed-integer non-linear programming (MINLP) problem.
- To solve the original problem, we discuss the two cases of delay minimization and energy consumption minimization respectively. We propose an optimization method based on a combination of penalty-based successive convex approximation (SCA) method and alternate optimization algorithm to solve the problems. In order to obtain a better initial solution, we propose an initialization algorithm based on load balancing and Floyd's shortest path method.
- We verify the convergence and performance of the proposed algorithm. Numerical simulation results show that our proposed algorithm can converge quickly. In addition, through comparison with other three baseline schemes, it is proved that the proposed algorithm can achieve lower total delay and lower user energy consumption.

The rest of this paper is organized as follows: In Section 2, we introduce the related works of this paper. Section 3 presents the system model of LEO satellite-enhanced mobile edge computing network, including network model, communication model and computation model. The optimization problem is formulated in Section 4. Section 5 introduces the proposed optimization algorithm for overall delay minimization and energy consumption minimization. Section 6 shows the simulation results. We summarize the paper in Section 7.

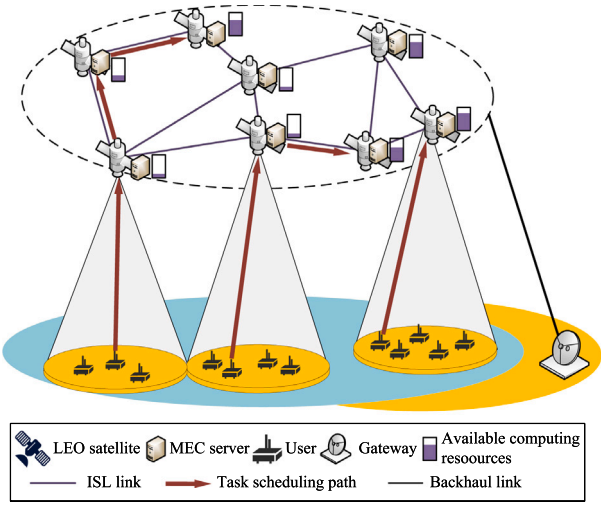


Fig. 1. The scenario of LEO satellite-enhanced mobile edge computing network.

2. Related works

At present, many research results on multi-dimensional resource scheduling technology for inter-satellites and satellite–terrestrial have been achieved. Below, we introduce the research progress of inter-satellite resource scheduling technology and satellite–terrestrial resource scheduling technology respectively.

Inter-satellite resource scheduling technology studies the path selection and transmission bandwidth allocation mechanism for data transmission between multiple satellites, including single source node-single destination node data transmission and resource scheduling, and multi-source node-multi-destination node data transmission. and resource scheduling. X. Jia et al. [15] proposed using ISLs for data transmission between satellites and earth stations (ESs) to increase the amount of data transmitted between satellites and ESs. The authors maximized the data download volume of ESs by alternately optimizing data transmission between satellites and data transmission between satellites and ESs. M. M. Gost. et al. [16] minimized energy consumption of all LEO satellites by optimizing the communication resources of the ISL links and the computing resources of edge computing nodes while meeting the computing delay constraints of all tasks. In [17], Yunhui Huang et al. proposed a network status representation method for LEO satellite network based on pheromone and ant colony algorithm. In order to improve the utility of the transmission path, the authors proposed a multi-path traffic scheduling strategy based on reinforcement learning method. Youjian Zou et al. [18] investigated path optimization methods under two-layer satellite networks to improve satellite network data transmission efficiency and reduce network switching complexity.

We summarize representative studies related to satellite–terrestrial edge computing in Table 1. As shown in the table, although extensive work has explored computation offloading and resource scheduling in satellite–terrestrial integrated networks, research on inter-satellite offloading, particularly multi-hop offloading over inter-satellite links, remains relatively limited. In [19], Qingqing Tang et al. studied a satellite network architecture that supports cloud computing and edge computing. The alternating direction method for multipliers (ADMMs) algorithm is designed to reduce the total energy consumption of users, while satisfying the task processing delay and node computing capacities. Zhu et al. [20] investigated how to jointly optimize the computing and communication resources, and task offloading strategies of satellite–terrestrial integrated networks. A joint optimization algorithm of closed-form scheme and particle swarm optimization is

Table 1
Overview of recent researches.

Ref.	Inter-satellite offloading	Multi-hop offloading	Delay optimization	Energy optimization	Algorithm
[19]	×	×	×	✓	ADMMs algorithm
[20]	×	×	✓	×	Particle swarm algorithm
[21]	×	×	✓	✓	Attention mechanism and PPO collaborative algorithm
[23]	×	×	✓	✓	discrete whale optimization algorithm
[24]	×	×	×	✓	Hierarchical game algorithm
[25]	✓	×	✓	✓	DRL combining SD3 algorithm
[26]	×	×	×	✓	MADDPG algorithm
[27]	✓	×	×	✓	Multi-agent DRL algorithm
[28]	×	×	✓	×	Digital twin-driven semantic offloading and PPO algorithm
[29]	×	×	✓	×	Federated learning algorithm

used to minimize system delay. In [21], authors studied the multi-task offloading problem for satellite user networks to improve the efficiency of multi-task offloading. Cao et al. [11] minimized the energy consumption of the system for the LEO satellite–terrestrial networks while satisfying the computing delay constraints of all tasks, and the energy consumption constraints of all computing nodes. In [22], by using rate-splitting multiple access (RSMA) as the modulation method for communication between satellites and ground nodes, the authors studied the problem of maximizing secrecy energy efficiency under RSMA-based satellite–terrestrial converged networks. In [23], latency and energy optimization in LEO edge computing is investigated, and a discrete whale optimization algorithm is proposed for joint offloading and resource allocation. P. Li et al. [24] investigated joint task offloading and resource allocation in hybrid MEC enabled LEO satellite networks and proposed an iterative optimization algorithm to reduce system delay and energy consumption. In [25], resource management for task inference in LEO satellite networks is investigated, and a deep reinforcement learning (DRL) based algorithm is designed to improve latency and energy performance. H. Li et al. [26] designed a multi-agent DRL algorithm (MADDPG) to jointly optimize task offloading and resource allocation in LEO MEC networks, aiming to minimize the total energy consumption while meeting latency constraints. In [27], collaborative task offloading in satellite MEC networks is studied, and a multi-agent DRL algorithm is proposed to optimize offloading and resource allocation. In [28], the authors designed a digital-twin-driven semantic offloading framework for LEO-MEC, jointly optimizing offloading and power to improve delay and energy performance. Han et al. [29] studied cooperative federated learning over ground–satellite integrated networks, jointly optimizing local update selection and communication–computing resources to reduce training delay under energy constraints.

3. System model

In this section, we first introduce the model of satellites collaborative computing network, including its topology and specific components. Furthermore, we provide separate introductions to the computation model and the communication model in this satellites collaborative computing network.

3.1. Network model

We consider the LEO satellite-enabled mobile computing network as shown in Fig. 1. The network comprises of a gateway on the ground, M LEO satellites and N terrestrial users. The terrestrial gateway is connected with LEO satellites through Ka-band wireless link [20], the transmission rate of this wireless link is very high, ranging from several gigabytes to tens of terabytes [30]. N terrestrial users are distributed at different locations on the ground, and each user can transmit task information and task data to LEO satellite by OFDMA technique [31].

The functions of terrestrial gateway include two aspects: (1) It receives information from LEO satellites and terrestrial users, including the computing resource information of each server and user's CPU, communication resource information between nodes, task information of user devices, etc. (2) Based on the collected information, it formulates optimized path scheduling and resource allocation strategy, and distributes the strategy to each node in the LEO satellite-enabled mobile computing network.

There are M LEO satellites in this system, expressed as $\mathcal{M} = \{1, 2, \dots, M\}$. Each LEO satellite can communicate with terrestrial users through OFDMA technology [32]. And each LEO satellite carries an edge computing server. Therefore, LEO satellites can process tasks offloaded by terrestrial users on their own servers. In addition, LEO satellite can also forward terrestrial users' tasks to other LEO satellites through ISL.

We denote the collection of N terrestrial users as $\mathcal{N} = \{1, 2, \dots, N\}$, each user is equipped with an antenna. We use a tuple $\{D_n, c_n, T_n\}$ to denote the task generated by user n, where D_n represents the data size of the task (unit: bit), c_n represents the number of CPU cycles required to process 1 bit of the task, and T_n represents the maximum tolerant delay of the task (unit: second).

In practical LEO systems, satellites continuously move along their orbits, which leads to time-varying network topology and link conditions. In this paper, we adopt a quasi-static modeling framework commonly used in LEO MEC systems. Specifically, the time horizon is divided into short scheduling slots. Since the movement of satellites within one slot is relatively small compared with the time scale of computation offloading and resource allocation, the satellite positions, coverage relationships, and channel states are assumed to remain unchanged during each slot, while they may vary across different slots. This assumption allows us to focus on the optimization of offloading and path scheduling under a given network snapshot, while still capturing the long-term impact of satellite mobility through slot-to-slot topology updates.

3.2. Computation model

In this paper, the task can be processed locally on the user device, or offloaded to the LEO satellite and processed on the edge computing server of LEO satellite. In the following, we describe these two computation model in detail.

3.2.1. Local computation model

Let $\alpha_n \in [0, 1]$ represents the ratio of the data size offloaded to the satellite by user device n to the total task data size. Therefore, the data size of task processed locally by user n is $(1 - \alpha_n)D_n$, and the number of CPU cycles required to process is $(1 - \alpha_n)D_n c_n$.

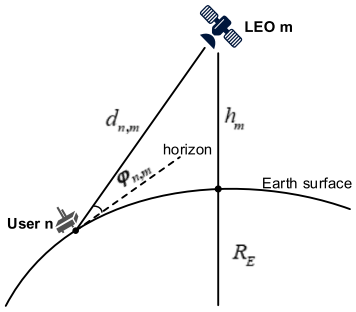


Fig. 2. Distance model between terrestrial user and LEO satellite.

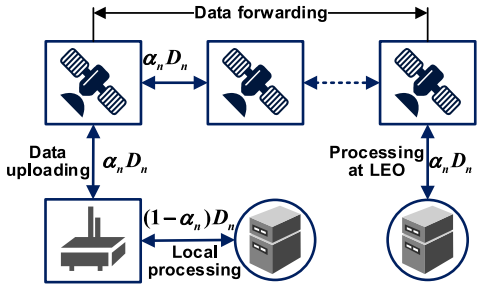


Fig. 3. Task procession procedure.

3.2.2. LEO satellites computation model

If user device n is accessed to LEO satellite m, we denote it as $\beta_{n,m} = 1$, otherwise, $\beta_{n,m} = 0$. When LEO satellite m receives task n, it can process it on its server, expressed as $\eta_{n,m} = 1$, otherwise, $\eta_{n,m} = 0$. In addition, LEO satellite m can also forward task n to other LEO satellite with ISL. When LEO satellite m forwards task n to LEO satellite m', we denote it as $\zeta^n_{m,m'} = 1$, otherwise, $\zeta^n_{m,m'} = 0$. During the aforementioned procedure, the size of the processed and forwarded data is $\alpha_n D_n$, and the number of CPU cycles required to process is $\alpha_n D_n c_n$.

3.3. Communication model

3.3.1. User-LEO satellite communication model

In this paper, when terrestrial user n is connected to LEO satellite m, the free space path loss in dB between user n and LEO m can be formulated as [33]

$$PL(d_{n,m}, f_c) = 32.45 + 20\log_{10} d_{n,m} + 20\log_{10} f_c \tag{1}$$

where f_c is the frequency of carrier signal between user device n and LEO satellite m. $d_{n,m}$ is the distance in meter between user n and LEO m. Because the user device is located on the ground, and the LEO satellite flies at an altitude of hundreds of kilometers, as shown in Fig. 2. the distance between user n and LEO m can be expressed as

$$d_{n,m} = \sqrt{R_E^2 \sin^2 \varphi_{n,m} + h_m^2 + 2h_m R_E} - R_E \sin \varphi_{n,m} \tag{2}$$

where R_E is the radium of Earth, which satisfies $R_E = 6371$ km [34]. $\varphi_{n,m}$ is the elevation angle from user n to LEO satellite m. And, h_m denotes the flight altitude of LEO satellite m above the ground.

We can denote the set of transmit power of all terrestrial users as $\mathcal{P} = \{p_{n,m} | n \in \mathcal{N}, m \in \mathcal{M}\}$, where $p_{n,m}$ denotes the transmit power from user n to LEO satellite m. Therefore, the channel capacity between user device n and LEO satellite m can be gotten based on the Shannon's Theorem, which is no less than achievable data rate between user n and LEO m [35].

$$R_{n,m} \le C_{n,m} = B_{n,m} \log_2(1 + \frac{p_{n,m} g_{n,m}}{B_{n,m} N_0}) \tag{3}$$

where $B_{n,m}$ represents the allocated bandwidth between user n and LEO m. $g_{n,m} = 10^{-0.1 \times PL(d_{n,m}, f_c)}$ is the channel gain between user n and LEO m. N_0 is the power density of additive white Gaussian noise (AWGN).

3.3.2. ISL communication model

The inter-satellite link capacity between LEO satellite m and LEO satellite m' can be expressed as $R^{ISL}_{m,m'}$, which is fixed in each time slot [10]. For each LEO satellite m, terrestrial user n's task can be processed on its server, i.e. $\eta_{n,m} = 1$, and $\sum_{m' \in S_m} \zeta^n_{m',m} - \sum_{m' \in S_m} \zeta^n_{m,m'} + \beta_{n,m} = 1$. If terrestrial user n's task is not processed on LEO satellite m's server, it can be denoted as $\eta_{n,m} = 0$, and $\sum_{m' \in S_m} \zeta^n_{m',m} - \sum_{m' \in S_m} \zeta^n_{m,m'} + \beta_{n,m} = 1$ [36]. Based on above discussion, we can get $\sum_{m' \in S_m} \zeta^n_{m',m} + \beta_{n,m} = \sum_{m' \in S_m} \zeta^n_{m,m'} + \eta_{n,m}$.

4. Problem formulation

4.1. Delay analysis

As shown in Fig. 3, the task processing delay is divided into local processing delay and task offloading delay. Moreover, the task offloading delay is composed of data upload delay, data forwarding delay and on-board processing delay.

4.1.1. Local processing delay

Let f^{loc}_n be the computation resource allocated by user n to the local task, therefore, the local processing delay of user n can be calculated as

$$t^{loc}_n = \frac{(1-\alpha_n) D_n c_n}{f^{loc}_n} \tag{4}$$

4.1.2. Task offloading delay

When user n offloads its task to LEO satellite m, the task upload delay depends on the size of transmitted data and the transmission rate, which can be formulated as

$$t^{up}_{n,m} = \frac{\alpha_n D_n}{R_{n,m}} = \frac{\alpha_n D_n}{B_{n,m} \log_2(1 + \frac{p_{n,m} g_{n,m}}{B_{n,m} N_0})} \tag{5}$$

Each user only accesses one satellite and uploads data to the satellite it accesses, therefore, the total data upload delay for user n can be gotten as

$$t^{up}_n = \sum_{m \in \mathcal{M}} \beta_{n,m} t^{up}_{n,m} \tag{6}$$

Task can be forwarded between LEO satellites, we denote the data forwarding rate of task n from LEO satellite m to LEO satellite m' as $r^n_{m,m'}$. Therefore, the total forwarding delay of task n can be expressed as

$$t^{fw}_n = \sum_{m \in \mathcal{M}} \sum_{m' \in S_m} \zeta^n_{m,m'} \frac{\alpha_n D_n}{r^n_{m,m'}} \tag{7}$$

where S_m is the set of LEO satellites with ISL with LEO satellite m.

After the task n is offloaded to the edge server of LEO satellite, the task processing delay can be formulated as

$$t^{leo}_{n,m} = \begin{cases} \frac{\alpha_n D_n c_n}{f_{n,m}}, & \eta_{n,m} = 1 \\ 0, & \text{Otherwise} \end{cases} \tag{8}$$

where $f_{n,m}$ is the computing frequency which allocated to task n by LEO satellite m's server. And the total computation delay of task n at LEO can be obtained as

$$t^{leo}_n = \sum_{m \in \mathcal{M}} \eta_{n,m} t^{leo}_{n,m} \tag{9}$$

The task offloading delay of task n can be denoted as

$$t^{off}_n = t^{up}_n + t^{fw}_n + t^{leo}_n \tag{10}$$

The processing delay of user n's task depends on the local processing delay and task offloading delay, so we can get the task processing delay of task n

$$t_n = \max\{t_n^{loc}, t_n^{off}\} \tag{11}$$

4.2. Energy consumption analysis

The energy consumption of user device can be divided into two parts, that are energy consumption for local computing and data upload.

4.2.1. Energy consumption of local computing

user devices' local computing energy consumption can be gotten based on dynamic voltage and frequency scaling (DVFS) [37], and which can be calculated as

$$E_n^{loc} = k_n (f_n^{loc})^2 (1-\alpha_n) D_n c_n \tag{12}$$

where k_n is the coefficient related to the chip architecture of user device n.

4.2.2. Energy consumption of data upload

For task n, the energy consumption of data upload to LEO satellite m can be obtained as

$$E_{n,m}^{up} = \begin{cases} \dfrac{p_{n,m}\alpha_n D_n}{R_{n,m}}, & \beta_{n,m} = 1 \\ 0, & \text{Otherwise} \end{cases} \tag{13}$$

Therefore, the data upload energy consumption of task n is

$$E_n^{up} = \sum_{m\in\mathcal{M}} E_{n,m}^{up} \tag{14}$$

During the entire task processing process, the energy consumed by user n is the sum of local processing energy consumption and data upload energy consumption, that is,

$$E_n = E_n^{loc} + E_n^{up} \tag{15}$$

4.3. Problem formulation

In this paper, we aim for minimizing the delay of all user devices, and the energy consumption of all user devices. The optimization problem of this paper can be formulated as

$$\mathcal{P}0 \quad \min_{\boldsymbol{\alpha},\boldsymbol{\eta},\boldsymbol{\zeta},\boldsymbol{P},\boldsymbol{F},\boldsymbol{R}} X_1 \sum_{n\in\mathcal{N}} t_n + X_2 \sum_{n\in\mathcal{N}} E_n \tag{16a}$$

$$\text{s.t.} \quad \mathbf{C1}: \ 0 \le p_{n,m} \le P_n, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{16b}$$

$$\mathbf{C2}: \ 0 \le f_n^{loc} \le F_n, \forall m \in \mathcal{N} \tag{16c}$$

$$\mathbf{C3}: \ 0 \le f_{n,m}, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{16d}$$

$$\mathbf{C4}: \ \sum_{n\in\mathcal{N}} f_{n,m} \le F_m, \forall m \in \mathcal{M} \tag{16e}$$

$$\mathbf{C5}: \ 0 \le r_{m,m'}^n, \forall m \in \mathcal{M}, m' \in S_m, n \in \mathcal{N} \tag{16f}$$

$$\mathbf{C6}: \ \sum_{n\in\mathcal{N}} (r_{m,m'}^n + r_{m',m}^n) \le R_{m,m'}^{ISL}, \quad \forall m \in \mathcal{M}, m' \in S_m \tag{16g}$$

$$\mathbf{C7}: \ 0 \le \alpha_n \le 1, \forall n \in \mathcal{N} \tag{16h}$$

$$\mathbf{C8}: \ \eta_{n,m} \in \{0,1\}, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{16i}$$

$$\mathbf{C9}: \ \sum_{m\in\mathcal{M}} \eta_{n,m} = 1, \forall n \in \mathcal{N} \tag{16j}$$

$$\mathbf{C10}: \ \zeta_{m,m'}^n \in \{0,1\}, \forall m \in \mathcal{M}, m' \in S_m, n \in \mathcal{N} \tag{16k}$$

$$\mathbf{C11}: \ \sum_{m'\in S_m} \zeta_{m',m}^n + \beta_{n,m} = \sum_{m'\in S_m} \zeta_{m,m'}^n + \eta_{n,m}, \quad \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{16l}$$

$$\mathbf{C12}: \ t_n \le T_n, \forall n \in \mathcal{N} \tag{16m}$$

where $\boldsymbol{\alpha} = \{\alpha_n | n \in \mathcal{N}\}$, $\boldsymbol{\eta} = \{\eta_{n,m} | n \in \mathcal{N}, m \in \mathcal{M}\}$, $\boldsymbol{\zeta} = \{\zeta_{m,m'}^n | n \in \mathcal{N}, m \in \mathcal{M}, m' \in S_m\}$ $\boldsymbol{P} = \{p_{n,m}^{ts} | n \in \mathcal{N}, m \in \mathcal{M}\}$, $\boldsymbol{F} = \{f_n | n \in \mathcal{N}\} \cup \{f_{n,m} | n \in \mathcal{N}, m \in \mathcal{M}\}$ and $\boldsymbol{R} = \{r_{m,m'}^n | m \in \mathcal{M}, m' \in \mathcal{S}_m, n \in \mathcal{N}\}$ are sets of task offloading ratio, target LEO satellite selection, task forwarding path selection, terrestrial users' transmit power, computing resource allocation, and data forwarding rate. Constraint **C1** restricts user devices' transmit power. **C2** indicates that allocated computation resource of each user is non-negative and cannot exceed the maximum local computing capacity. **C3** and **C4** indicate that the computing frequency allocated by LEO to user is non-negative, and the sum of allocated computation resources cannot exceed the maximum computing frequency of its edge computing server. **C5** indicates that the data forwarding rate of each task is non-negative. **C6** represents that the data rate allocated to all task cannot exceed the achievable data rate between LEO satellites. **C7** represents the partial task offloading decision. **C8** and **C10** state that target LEO satellite selection decisions, and task's data forwarding strategies are binary variables. **C12** guarantees each task is completed within the maximum tolerance delay.

In this paper, we consider two cases of delay minimization and energy consumption minimization, therefore, we have $X_1 \in \{0,1\}$, $X_2 \in \{0,1\}$ and $X_1 + X_2 = 1$. Although the objective function is written in a weighted form involving both latency and energy, this work focuses on two representative operating modes that frequently arise in LEO satellite networks. When the network mainly serves delay-sensitive applications such as real-time monitoring, disaster response, or interactive services, the primary system objective is to minimize end-to-end task completion time. In such scenarios, the controller tends to allocate more communication and computing resources to accelerate task processing, even at the cost of higher energy consumption. In contrast, when user devices are energy-limited (e.g., battery-powered terminals), reducing the energy consumed for computation and offloading becomes crucial. Therefore, under the energy-oriented mode, the optimization aims to minimize the total energy consumption of users, while tolerating moderate delay increases. Consequently, the weighting coefficients are restricted to two extreme values, corresponding to delay-oriented and user-energy-oriented optimization modes. These two configurations capture typical service demands and resource constraints widely encountered in practical LEO satellite MEC systems.

5. Proposed algorithm for joint path scheduling and multiple resource allocation

In this section, we introduce our solutions to the delay minimization and energy consumption minimization problems respectively.

5.1. Solution to delay minimization problem

When $X_1 = 1$ and $X_2 = 0$, the original optimization problem is the delay minimization problem, which can be formulated as

$$\mathcal{P}1: \quad \min_{\boldsymbol{\alpha},\boldsymbol{\eta},\boldsymbol{\zeta},\boldsymbol{P},\boldsymbol{F},\boldsymbol{R}} \sum_{n\in\mathcal{N}} t_n \tag{17a}$$

$$\text{s.t.} \quad (\mathbf{C1}) - (\mathbf{C12}) \tag{17b}$$

This problem is a MINLP problem, we propose a alternative optimization algorithm to simplify the solution of $\mathcal{P}1$. First, we decompose it to two subproblems, the first decomposed problem is used to solve the task splitting and resource allocation problem under a given path scheduling decision (i.e. $\mathcal{P}1.1$), and the second subproblem is used to get optimal path scheduling decision (i.e. $\mathcal{P}1.2$) .

5.1.1. Optimization of task splitting and resource allocation

With the fixed task scheduling strategy $\{\boldsymbol{\eta}, \boldsymbol{\zeta}\}$, the optimization of task splitting $\boldsymbol{\alpha}$ and resource allocation $\{\boldsymbol{P}, \boldsymbol{F}, \boldsymbol{R}\}$ problem can be formulated as

$$\mathcal{P}1.1: \quad \min_{\boldsymbol{\alpha},\boldsymbol{P},\boldsymbol{F},\boldsymbol{R}} \sum_{n\in\mathcal{N}} t_n \tag{18a}$$

$$\text{s.t.} \quad (\mathbf{C1}) - (\mathbf{C7}), (\mathbf{C12}) \tag{18b}$$

when the task splitting strategy is fixed, the problem $\mathcal{P}1.1$ can be deviled into two subproblems, which are local processing delay minimization and task offloading delay minimization. Therefore, we further decompose $\mathcal{P}1.1$ into resource allocation subproblem (i.e. $\mathcal{P}1.1.1$) and task splitting optimization subproblem (i.e. $\mathcal{P}1.1.2$).

The optimization of resource allocation can be expressed as

$$\mathcal{P}1.1.1: \quad \min_{\boldsymbol{P},\boldsymbol{F},\boldsymbol{R}} \sum_{n\in\mathcal{N}} \max\{t_n^{loc}, t_n^{off}\} \tag{19a}$$

$$\text{s.t.} \quad (\mathbf{C1}) - (\mathbf{C6}), (\mathbf{C12}) \tag{19b}$$

The delay of local processing in this subproblem is only related to computing frequency allocated to local processing task. From (4) we can know that t_n decreases with the increase of f_n^{loc}. At the same time, f_n^{loc} cannot exceed the local maximum calculation frequency F_n (Constraint **C2**), therefore, we can get the optimal local computing frequency satisfies

$$f_n^{loc,*} = F_n, \forall n \in \mathcal{N} \tag{20}$$

In addition, from (5) and constraint (**C1**), we can also get the optimal transmit power of each user device

$$p_{n,m}^* = P_n, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{21}$$

Now, we can reformulate the $\mathcal{P}1.1.1$ as

$$\min_{\{f_{n,m}\},\boldsymbol{R}} \sum_{n\in\mathcal{N}} \max\{t_n^{loc}, t_n^{off}\} \tag{22a}$$

$$\text{s.t.} \quad (\mathbf{C3}) - (\mathbf{C6}), (\mathbf{C12}), (20), (21) \tag{22b}$$

We introduce auxiliary variables $\boldsymbol{t} = \{t^1, t^2, \ldots, t^N\}$ to transform the above problem. And, the problem $\mathcal{P}1.1.1$ can be rewritten as

$$\min_{\{f_{n,m}\},\boldsymbol{R}} \sum_{n\in\mathcal{N}} t^n \tag{23a}$$

$$\text{s.t.} \quad \frac{(1-\alpha_n)D_n c_n}{f_n^{loc}} \le t^n, \forall n \in \mathcal{N} \tag{23b}$$

$$\sum_{m\in\mathcal{M}} \beta_{n,m} t_{n,m}^{up} + \sum_{m\in\mathcal{M}} \sum_{m'\in\mathcal{S}_m} \zeta_{m,m'}^n \frac{\alpha_n D_n}{r_{m,m'}^n} + \sum_{m\in\mathcal{M}} \eta_{n,m} t_{n,m}^{leo} \le t^n, \forall n \in \mathcal{N} \tag{23c}$$

$$t^n \le T_n, \forall n \in \mathcal{N} \tag{23d}$$

$$(\mathbf{C3}) - (\mathbf{C6}), (20), (21) \tag{23e}$$

Obviously, this is a convex problem, and we can solve it by CVX toolbox [38].

With the fixed resource allocation strategy, the optimization of task splitting can be expressed as

$$\mathcal{P}1.1.2: \quad \min_{\boldsymbol{\alpha}} \sum_{n\in\mathcal{N}} \max\{\frac{D_n c_n}{f_n^{loc,*}}(1-\alpha_n), t_n^{cof}\alpha_n\} \tag{24a}$$

$$\text{s.t.} \quad (\mathbf{C7}), (\mathbf{C12}) \tag{24b}$$

where $t_n^{cof} = \frac{D_n}{B_{n,m}\log_2(1+\frac{p_{n,m}^* g_{n,m}}{B_{n,m}N_0})} + \sum_{m\in\mathcal{M}}\sum_{m'\in\mathcal{S}_m} \zeta_{m,m'}^n \frac{D_n}{r_{m,m'}^{n,*}} + \sum_{m\in\mathcal{M}} \eta_{n,m} \frac{D_n c_n}{f_{n,m}}^*$. This problem is a linear programming problem. Based on martingale theory, we can get the optimal task splitting strategy. As shown in Fig. 4, while local processing delay equal to task offloading delay, the task processing delay is minimal. The optimal task splitting ratio can be obtained as

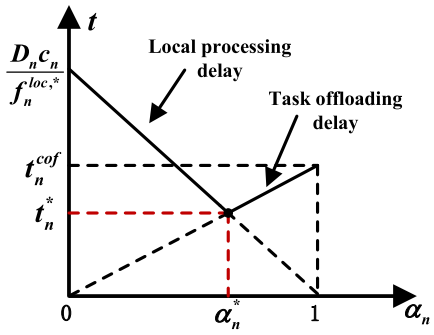


Fig. 4. Relationship between task processing delay and task splitting ratio.

$$\alpha_n^* = \frac{\frac{D_n c_n}{f_n^{loc,*}}}{\frac{D_n c_n}{f_n^{loc,*}} + t_n^{cof}}, \forall n \in \mathcal{N} \tag{25}$$

5.1.2. Optimization of path scheduling

With the fixed task splitting and resource allocation strategy $\{\boldsymbol{\alpha}, \boldsymbol{P}, \boldsymbol{F}, \boldsymbol{R}\}$, the path scheduling optimization problem can be organized as

$$\mathcal{P}1.2: \quad \min_{\boldsymbol{\eta},\boldsymbol{\zeta}} \sum_{n\in\mathcal{N}} t_n \tag{26a}$$

$$\text{s.t.} \quad (\mathbf{C8}) - (\mathbf{C12}) \tag{26b}$$

First, we relax $\boldsymbol{\eta}$ and $\boldsymbol{\zeta}$ into continuous variables, then **C8** and **C10** are converted to

$$\eta_{n,m} \in [0,1], \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{27a}$$

$$\eta_{n,m}(1-\eta_{n,m}) \le 0, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{27b}$$

and

$$\zeta_{m,m'}^n \in [0,1], \forall m \in \mathcal{M}, m' \in \mathcal{S}_m, n \in \mathcal{N} \tag{28a}$$

$$\zeta_{m,m'}^n(1-\zeta_{m,m'}^n) \le 0, \forall m \in \mathcal{M}, m' \in \mathcal{S}_m, n \in \mathcal{N} \tag{28b}$$

We add constraints (27b) and (28b) as penalty terms to the objective function, and perform first-order Taylor expansion on the objective function. Problem $\mathcal{P}1.2$ can be rewritten as

$$\begin{aligned}\mathcal{P}1.2a: \quad \min_{\boldsymbol{\eta},\boldsymbol{\zeta}} & \sum_{n\in\mathcal{N}} t_n + \mu \sum_{m\in\mathcal{M}}\sum_{n\in\mathcal{N}} (\eta_{n,m} - {\eta'_{n,m}}^2 \\ & - 2\eta'_{n,m}(\eta_{n,m} - \eta'_{n,m})) \\ & + \mu \sum_{m\in\mathcal{M}}\sum_{m'\in\mathcal{S}_m}\sum_{n\in\mathcal{N}} (\zeta_{m,m'}^n - {\zeta_{m,m'}^n}'^2 \\ & - 2\zeta_{m,m'}^n{}'(\zeta_{m,m'}^n - \zeta_{m,m'}^n{}')) \end{aligned} \tag{29a}$$

$$\text{s.t.} \quad (\mathbf{C9}), (\mathbf{C11}), (\mathbf{C12}), (27a), (28a) \tag{29b}$$

where $\boldsymbol{\eta}' = \{\eta_{n,m} | n \in \mathcal{N}, m \in \mathcal{M}\}$ and $\boldsymbol{\zeta}' = \{\zeta_{m,m'}^n | m \in \mathcal{M}, m' \in \mathcal{S}_m, n \in \mathcal{N}\}$ are collections of feasible solutions during algorithm iteration. μ is the penalty coefficient. As the number of iterations of the algorithm increases, μ continues to increase. $\mathcal{P}1.2a$ is convex which can be solved by CVX toolbox. We summarize the solution method of problem $\mathcal{P}1.2$ as shown in **Algorithm** 1.

Based on the discussion above, a summary of the optimization algorithm based on block coordinate descent is presented in **Algorithm** 2. Resource allocation and path scheduling are iteratively optimized in an alternating manner. The input for the resource allocation optimization subproblem is the optimized path scheduling strategy, and the input for the path scheduling optimization subproblem is the optimized resource allocation strategy. By iteratively solving these two subproblems

Algorithm 1 Optimization of Path Scheduling for Delay Minimization

Input: Initialized penalty term coefficient μ. Penalty term coefficient growth rate r_μ. Iteration number indicator parameter $i = 0$. The maximum number of iterations I. Task splitting and resource allocation strategy $\{\boldsymbol{\alpha}, \boldsymbol{P}, \boldsymbol{F}, \boldsymbol{R}\}$. Initialized path scheduling strategy $\{\boldsymbol{\eta}(0), \boldsymbol{\zeta}(0)\}$.
do
  Calculate $\{\boldsymbol{\eta}(i+1), \boldsymbol{\zeta}(i+1)\}$ based on $\{\boldsymbol{\eta}(i), \boldsymbol{\zeta}(i)\}$ by solving problem $\mathcal{P}1.2a$.
  $i = i + 1$.
  $\boldsymbol{\eta}^* = \boldsymbol{\eta}(i)$, $\boldsymbol{\zeta}^* = \boldsymbol{\zeta}(i)$.
  $\mu = r_\mu \times \mu$.
while $i < I$ and $\left|\frac{\sum_{n\in\mathcal{N}} t_n(i) - \sum_{n\in\mathcal{N}} t_n(i-1)}{\sum_{n\in\mathcal{N}} t_n(i-1)}\right| \geq \epsilon$
Output: Optimized path scheduling strategy $\{\boldsymbol{\eta}^*, \boldsymbol{\zeta}^*\}$

alternately until convergence of the algorithm, we can obtain optimized path scheduling and resource allocation strategies.

Clearly, the proper functioning of **Algorithm 2** necessitates a feasible initial path scheduling strategy. A well-designed initial solution aids the algorithm in converging towards a superior local optimum. Hence, leveraging load balancing and shortest path strategies, we devised an initialization algorithm for path scheduling, as outlined in **Algorithm 3**. In this algorithm, we first utilize the Floyd algorithm to obtain the shortest hop count and path information between each pair of satellites (Line 2) [39]. Then, based on the data size, we sequentially select the target satellite for offloading for each task (Line 5 to Line 14). Finally, leveraging the path information and target satellites, we obtain the scheduling paths for each task.

Algorithm 2 Joint Path Scheduling and Resource Allocation for Overall Delay Optimization ($\mathcal{P}1$)

Input: Initialized feasible path scheduling strategy $\{\boldsymbol{\eta}(0), \boldsymbol{\zeta}(0)\}$. The maximum number of iterations I_{max}. Iteration number indicator parameter $i = 0$.
while $i \leq I_{max}$ **do**
  Calculate $\{f_n^{loc}\}(i)$ and $\{p_{n,m}\}(i)$ based on (20) and (21).
  Calculate $\{f_{n,m}\}(i)$ and $\boldsymbol{R}(i)$ by solved the problem (23a).
  Calculate $\boldsymbol{\alpha}(i)$ based on (25).
  Obtain the total delay of the system $t_{sum}(i)$
  if $i \geq 1$ and $\left|\frac{t_{sum}(i) - t_{sum}(i-1)}{t_{sum}(i-1)}\right| \leq \epsilon$ **do**
    break.
  end if
  $i = i + 1$
  Obtain the optimized path scheduling strategy $\boldsymbol{\eta}(i)$ and $\boldsymbol{\zeta}(i)$ through **Algorithm 1**.
end while
Output: Optimal path scheduling and resource allocation strategy $\{\boldsymbol{\alpha}(i), \boldsymbol{\eta}(i), \boldsymbol{\zeta}(i), \boldsymbol{P}(i), \boldsymbol{F}(i), \boldsymbol{R}(i)\}$.

5.2. Solution to energy consumption minimization problem

When $X_1 = 0$ and $X_2 = 1$, the original optimization problem is energy consumption minimization problem, we can be denoted as

$$\mathcal{P}2 \quad \min_{\boldsymbol{\alpha},\boldsymbol{\eta},\boldsymbol{\zeta},\boldsymbol{P},\boldsymbol{F},\boldsymbol{R}} \sum_{n\in\mathcal{N}} E_n \tag{30a}$$

$$\text{s.t.} \quad (\mathbf{C1}) - (\mathbf{C12}) \tag{30b}$$

Similar to solution of $\mathcal{P}1$, we can also decompose $\mathcal{P}2$ into two subproblems, task splitting and resource allocation subproblem ($\mathcal{P}2.1$), and path scheduling subproblem ($\mathcal{P}2.2$). Next, we introduce the solution methods of these two subproblems respectively.

Algorithm 3 Initialization Algorithm for Path Scheduling

Input: The input data size for all tasks $\{D_n | n \in \mathcal{N}\}$. The ISL matrix S_{mat} (where element $S_{mat}(m, m')$ indicates whether there exists an inter-satellite link between satellite m and satellite m'). Initialize the array of task numbers received by each satellite $tf \in \mathcal{R}^{1\times M}$, with each element initialized to 0.
Using the Floyd algorithm to obtain the minimum hop matrix H_{hop} and the shortest path information matrix P_{path} between satellites.
Sort the tasks in descending order based on the input data volume and obtain the sorting vector, S_{ort}.
Calculate the maximum number of tasks that can be offloaded to each satellite to satisfy $TG_{max} = \lceil \frac{N}{M} \rceil$
for $i = 1$ to N **do**
  for $j = 1$ to M **do**.
    Based on the matrix H_{hop} and $\boldsymbol{\beta}$ obtain the satellite m which represents the jth satellite with the smallest number of hops to task i.
    if $tf(m) \geq TG_{max}$ **do**
      continue;
    end if
    $tf(m) = tf(m) + 1$.
    $\eta_{m,i} = 1$.
  end for
end for
Based on $\boldsymbol{\eta}$ and P_{path} obtain the initial path selection strategy $\boldsymbol{\zeta}$
Output: Initialized path scheduling strategies $\boldsymbol{\eta}$ and $\boldsymbol{\zeta}$.

5.2.1. Optimization of task splitting and resource allocation

When the path scheduling strategies $\{\boldsymbol{\eta}, \boldsymbol{\zeta}\}$ are fixed, the problem $\mathcal{P}2$ can be formulated as

$$\mathcal{P}2.1: \min_{\boldsymbol{\alpha},\boldsymbol{P},\boldsymbol{F},\boldsymbol{R}} \sum_{n\in\mathcal{N}} k_n (f_n^{loc})^2 (1-\alpha_n) D_n c_n + \sum_{n\in\mathcal{N}} \sum_{m\in\mathcal{M}} \frac{p_{n,m}\alpha_n D_n}{R_{n,m}} \tag{31a}$$

$$\text{s.t.} \quad (\mathbf{C1}) - (\mathbf{C7}), (\mathbf{C12}) \tag{31b}$$

We decompose the problem $\mathcal{P}2.1$ into three subproblems: The optimization of local computing resource allocation ($\mathcal{P}2.1.1$), the optimization of other computing and communication resource allocation ($\mathcal{P}2.1.2$) and the optimization of task splitting ratio ($\mathcal{P}2.1.3$). Below, we introduce the solutions to the three subproblems respectively.

The optimization of local computing resource allocation can be formulated as

$$\mathcal{P}2.1.1 \quad \min_{\{f_n^{loc}\}} \sum_{n\in\mathcal{N}} (1-\alpha_n)(f_n^{loc})^2 D_n c_n \tag{32a}$$

$$\text{s.t.} \quad \frac{(1-\alpha_n) D_n c_n}{f_n^{loc}} \leq T_n, \forall n \in \mathcal{N} \tag{32b}$$

$$(\mathbf{C2}) \tag{32c}$$

This is a linear programming problem. From constraints (**C2**) and (32b), we can get $f_n^{loc} \in \left[\frac{(1-\alpha_n) D_n c_n}{T_n}, F_n\right]$. Therefore, the optimal local processing resource can be calculated as

$$f_n^{loc,*} = \frac{(1-\alpha_n) D_n c_n}{T_n} \tag{33}$$

The optimization of transmit power, data forwarding rate, and computing resource allocation of satellite servers is coupled to each other, so we optimize these three types of variables together. With fixed task splitting strategy and local processing resource allocation, the problem $\mathcal{P}2.1$ can be reformulated as

$$\mathcal{P}2.1.2 \quad \min_{\boldsymbol{P},\{f_{n,m}\},\boldsymbol{R}} \sum_{n\in\mathcal{N}} \sum_{m\in\mathcal{M}} \frac{p_{n,m}\alpha_n D_n}{R_{n,m}} \tag{34a}$$

$$\text{s.t.} \quad (\mathbf{C1}), (\mathbf{C3}) - (\mathbf{C6}) \tag{34b}$$

$$\sum_{m\in\mathcal{M}} \beta_{n,m} \frac{\alpha_n D_n}{B_{n,m}\log_2(1+\frac{p_{n,m}g_{n,m}}{B_{n,m}N_0})} + \sum_{m\in\mathcal{M}}\sum_{m'\in S_m} \zeta^n_{m,m'} \frac{\alpha_n D_n}{r^n_{m,m'}} + \sum_{m\in\mathcal{M}} \eta_{n,m}\frac{\alpha_n D_n c_n}{f_{n,m}} \le T_n, \forall n \in \mathcal{N} \tag{34c}$$

This is a non-convex problem. By introducing variable $t^b_{n,m} = \frac{1}{\log_2(1+\frac{p_{n,m}g_{n,m}}{B_{n,m}N_0})}$ and transforming it into a convex optimization problem. And then, we can solve it to obtain the optimal solution for this problem.

With the fixed resource allocation strategy $\{\boldsymbol{P}, \boldsymbol{F}, \boldsymbol{R}\}$, the optimization of task splitting ratio in this problem is given as

$\mathcal{P}2.1.3$

$$\min_{\boldsymbol{\alpha}} \sum_{n\in\mathcal{N}} (k_n (f^{loc}_n)^2 (1-\alpha_n) D_n c_n + \sum_{m\in\mathcal{M}} \frac{p_{n,m}\alpha_n D_n}{R_{n,m}}) \tag{35a}$$

$$\text{s.t.} \quad (\mathbf{C7}), (\mathbf{C12}) \tag{35b}$$

According to constraints (**C7**) and (**C12**), we have $\alpha_n \in \left[\max\{0, 1-\frac{T_n f^{loc,*}_n}{D_n c_n}\}, \min\{1, \frac{T_n}{t^{cof}_n}\}\right]$. When we only consider the objective function and constraint (**C7**), similar to the solution of problem $\mathcal{P}1.1.2$, the optimal task splitting ratio is $\alpha'_n = \frac{k_n (f^{loc,*}_n)^2 D_n c_n}{k_n (f^{loc,*}_n)^2 D_n c_n + \sum_{m\in\mathcal{M}} \frac{p_{n,m} D_n}{R_{n,m}}}$. Therefore, the optimal task splitting ratio of problem $\mathcal{P}2.1.3$ is

$$\alpha^*_n = \begin{cases} \frac{T_n}{t^{cof}_n}, & \frac{T_n}{t^{cof}_n} \le \alpha'_n \\ \alpha'_n, & 1-\frac{T_n f^{loc,*}_n}{D_n c_n} < \alpha'_n < \frac{T_n}{t^{cof}_n} \\ 1-\frac{T_n f^{loc,*}_n}{D_n c_n}, & \alpha'_n \le 1-\frac{T_n f^{loc,*}_n}{D_n c_n} \end{cases} \tag{36}$$

Partitioning the variables into three interdependent groups yields three subproblems. Building on this premise, fixing two groups of variables allows us to optimize the solution for the third group. Through iterative steps among these groups, we derive an optimized strategy for task segmentation and resource allocation, as outlined in **Algorithm** 4.

Algorithm 4 Task Splitting and Resource Allocation for Energy Consumption Optimization

Input: Path scheduling strategy $\{\boldsymbol{\eta}, \boldsymbol{\zeta}\}$. Maximum number of iterations I_{max}. Maximum allowable error tolerance δ. The initialized task splitting and resource allocation strategy $\{\boldsymbol{\alpha}(0), \boldsymbol{P}(0), \boldsymbol{F}(0), \boldsymbol{R}(0)\}$. Total energy consumption $E_{sum}(0)$. $i = 1$.
while $i \le I_{max}$ **do**
  Obtain $\{f^{loc}_n\}(i)$ via (33) at $\boldsymbol{\alpha}(i-1)$, $\boldsymbol{P}(i-1)$, $\boldsymbol{R}(i-1)$, and $\{f_{n,m}\}(i-1)$.
  Obtain $\boldsymbol{P}(i)$, $\boldsymbol{R}(i)$, and $\{f_{n,m}\}(i)$ via solving problem $\mathcal{P}2.1.2$ at $\{f^{loc}_n\}(i)$ and $\boldsymbol{\alpha}(i-1)$.
  Obtain $\boldsymbol{\alpha}(i)$ via (36) at $\{\boldsymbol{P}(i), \boldsymbol{F}(i), \boldsymbol{R}(i)\}$.
  Calculate total energy consumption $E_{sum}(i)$ based on $\{\boldsymbol{\alpha}(i), \boldsymbol{P}(i), \boldsymbol{F}(i), \boldsymbol{R}(i)\}$.
  if $\left|\frac{E_{sum}(i)-E_{sum}(i-1)}{E_{sum}(i-1)}\right| \le \delta$ **do**
    break.
  end if
  Update the optimal task splitting and resource allocation strategy $\{\boldsymbol{\alpha}^*, \boldsymbol{P}^*, \boldsymbol{F}^*, \boldsymbol{R}^*\} = \{\boldsymbol{\alpha}(i), \boldsymbol{P}(i), \boldsymbol{F}(i), \boldsymbol{R}(i)\}$.
  $i = i + 1$
end while
Output: Optimal task splitting and resource allocation strategy $\{\boldsymbol{\alpha}^*, \boldsymbol{P}^*, \boldsymbol{F}^*, \boldsymbol{R}^*\}$.

5.2.2. Optimization of path scheduling

We need to make a optimal path scheduling strategy based on fixed task splitting and multiple resource allocation decision for energy minimization. First, we can write the path scheduling subproblem for energy minimization as

$$\mathcal{P}2.2 \quad \min_{\boldsymbol{\eta},\boldsymbol{\zeta}} \sum_{n\in\mathcal{N}} E_n \tag{37a}$$

$$\text{s.t.} \quad (\mathbf{C8}) - (\mathbf{C12}) \tag{37b}$$

The solution to this problem is the same as the solution to problem $\mathcal{P}1.2$. The problem is also solved iteratively starting from a feasible path scheduling strategy, and the iteration stops when the algorithm converges to a local optimal solution. At each iteration, problem $\mathcal{P}2.2$ can be rewritten as

$$\begin{aligned} \mathcal{P}2.2a: \quad & \min_{\boldsymbol{\eta},\boldsymbol{\zeta}} \sum_{n\in\mathcal{N}} E_n + \mu \sum_{m\in\mathcal{M}}\sum_{n\in\mathcal{N}} (\eta_{n,m} - \eta'^2_{n,m} \\ & - 2\eta'_{n,m}(\eta_{n,m} - \eta'_{n,m})) \\ & + \mu \sum_{m\in\mathcal{M}}\sum_{m'\in S_m}\sum_{n\in\mathcal{N}} (\zeta^n_{m,m'} - \zeta^{n\prime 2}_{m,m'} \\ & - 2\zeta^{n\prime}_{m,m'}(\zeta^n_{m,m'} - \zeta^{n\prime}_{m,m'})) \end{aligned} \tag{38a}$$

$$\text{s.t.} \quad (\mathbf{C9}), (\mathbf{C11}), (\mathbf{C12}), (27a), (28a) \tag{38b}$$

Similar to **Algorithm** 1, we can iteratively solve Problem $\mathcal{P}2.2a$ to obtain the optimized solution for Problem $\mathcal{P}2.2$, thereby deriving an optimized path scheduling strategy. By iteratively solving problems $\mathcal{P}2.1$ and $\mathcal{P}2.2$ until convergence, we can obtain an optimized task offloading and resource allocation strategy for minimizing energy consumption. The specific algorithm is detailed in **Algorithm** 5.

Algorithm 5 Path Scheduling and Resource Allocation for Energy Consumption Optimization ($\mathcal{P}2$)

Input: Maximum allowable error tolerance δ. The maximum number of iterations I_{max}. Iteration number indicator parameter $i = 0$.
Obtain a initial feasible path scheduling strategy $\{\boldsymbol{\eta}(0), \boldsymbol{\zeta}(0)\}$ based on **Algorithm** 3.
while $i \le I_{max}$ **do**
  Obtain $\{\boldsymbol{P}(i), \boldsymbol{F}(i), \boldsymbol{R}(i)\}$ via **Algorithm** 4 at $\{\boldsymbol{\eta}(i), \boldsymbol{\zeta}(i)\}$.
  Calculate $E_{sum}(i)$ based on $\{\boldsymbol{\eta}(i), \boldsymbol{\zeta}(i), \boldsymbol{P}(i), \boldsymbol{F}(i), \boldsymbol{R}(i)\}$.
  Update the optimal task splitting and resource allocation strategy $\{\boldsymbol{\eta}^*, \boldsymbol{\zeta}^*, \boldsymbol{\alpha}^*, \boldsymbol{P}^*, \boldsymbol{F}^*, \boldsymbol{R}^*\} = \{\boldsymbol{\eta}(i), \boldsymbol{\zeta}(i), \boldsymbol{\alpha}(i), \boldsymbol{P}(i), \boldsymbol{F}(i), \boldsymbol{R}(i)\}$.
  if $i \ge 1$ and $\left|\frac{E_{sum}(i)-E_{sum}(i-1)}{E_{sum}(i-1)}\right| \le \delta$ **do**
    break.
  end if
  $i = i + 1$.
  Obtain $\{\boldsymbol{\eta}(i), \boldsymbol{\zeta}(i)\}$ at $\{\boldsymbol{P}(i-1), \boldsymbol{F}(i-1), \boldsymbol{R}(i-1)\}$
end while
Output: Optimal path scheduling and resource allocation strategy $\{\boldsymbol{\eta}^*, \boldsymbol{\zeta}^*, \boldsymbol{\alpha}^*, \boldsymbol{P}^*, \boldsymbol{F}^*, \boldsymbol{R}^*\}$.

5.3. The complexity analysis

Since both the delay minimization and the energy minimization algorithms rely on the initialization procedure in **Algorithm** 3, we first analyze the complexity of this initialization algorithm. Specifically, this algorithm first computes the shortest paths among the M satellites using the Floyd algorithm, whose complexity is $\mathcal{O}(M^3)$. It then sorts the task data sizes of N users, which has a complexity lower bound of $\mathcal{O}(N \log N)$. Finally, it performs a double loop over all users and satellites to construct the initial path scheduling, which incurs a complexity of $\mathcal{O}(NM)$. Therefore, the overall complexity of the initialization algorithm is $\mathcal{O}(M^3 + N \log N + NM)$.

Complexity of the delay minimization algorithm: Based on the above result, we next analyze the complexity of the delay minimization algorithm. In each outer iteration, Algorithm 2 successively solves subproblems $\mathcal{P}1.1$ and $\mathcal{P}1.2$. For $\mathcal{P}1.1$, the original problem is finally reformulated as (23a), which is solved by an interior-point method. Let $I_1 = NM + V$ denote the number of decision variables in (23a), where V is the number of data forwarding rate variables, and let $I_2 = NM + M + V$ denote the number of constraints. According to the complexity analysis of interior-point methods in [11], the computational complexity of solving (23a) is on the order of $\mathcal{O}\big((I_1^2 I_2 + I_1^3) I_2^{0.5}\big)$. Similarly, for the path scheduling subproblem $\mathcal{P}1.2$, the corresponding convex subproblem ($\mathcal{P}1.2a$) contains $I_3 = NM + V$ variables and $I_4 = 2NM + 2N + V$ constraints. $\mathcal{P}1.2$ is solved by a penalty-based iterative procedure, and we denote the number of inner iterations by I_{inner}. Thus, the complexity of solving $\mathcal{P}1.2$ is $\mathcal{O}\big(I_{\text{inner}}(I_3^2 I_4 + I_3^3) I_4^{0.5}\big)$. Let I_{outer} be the number of outer iterations of **Algorithm** 1. Then, the overall complexity of the proposed delay minimization algorithm can be expressed as $\mathcal{O}\Big(I_{\text{outer}}\big((I_1^2 I_2 + I_1^3) I_2^{0.5} + I_{\text{inner}}(I_3^2 I_4 + I_3^3) I_4^{0.5}\big)\Big)$. Combining this with the initialization algorithm, the total computational complexity of solving the delay minimization problem is $\mathcal{O}\Big(M^3 + N \log N + NM + I_{\text{outer}}\big((I_1^2 I_2 + I_1^3) I_2^{0.5} + I_{\text{inner}}(I_3^2 I_4 + I_3^3) I_4^{0.5}\big)\Big)$.

Complexity of the energy consumption minimization algorithm: Similarly to the delay minimization problem, the complexity of the energy consumption minimization algorithm is determined by solving subproblems $\mathcal{P}2.1$ and $\mathcal{P}2.2$ in each outer iteration. For $\mathcal{P}2.1$, the reformulated convex problem contains $I_5 = 2NM + V$ decision variables and $I_6 = 2NM + N + M + V$ constraints. When an interior-point method is employed, the computational complexity of solving $\mathcal{P}2.1$ is on the order of $\mathcal{O}\big((I_5^2 I_6 + I_5^3) I_6^{0.5}\big)$. The path scheduling subproblem $\mathcal{P}2.2$ has the same structure as $\mathcal{P}1.2$ in the delay minimization algorithm. Therefore, the total computational complexity of solving the energy minimization problem is given by $\mathcal{O}\Big(M^3 + N \log N + NM + I_{\text{outer}}\big((I_5^2 I_6 + I_5^3) I_6^{0.5} + I_{\text{inner}}(I_3^2 I_4 + I_3^3) I_4^{0.5}\big)\Big)$.

6. Simulation results

In this section, we designed simulation experiments to verify the performance of the path scheduling and resource allocation optimization algorithms proposed in this paper. We conducted multiple sets of experiments to validate the optimization performance for both latency minimization and energy consumption minimization problems. The numerical results of the experiments are presented below.

6.1. Simulation settings

The simulation in this paper was implemented using MATLAB R2020b. We utilize STK 11.6 to obtain the position information of the satellite. By importing the ephemeris information of the OneWeb satellite constellation, we can obtain the precise position of the satellite. We consider a scenario of a satellites collaborative network comprising M LEO satellites (defaulting to 6) and 1 terrestrial gateway. Satellites fly at an altitude of 1200 km. There are N randomly distributed terrestrial user terminals (defaulting to 20) on the ground. Each user generates computation tasks whose input sizes are uniformly distributed in $[1, 2.2]$ Mbits. The number of CPU cycles required to process one bit data is 1000 [40,41]. And the maximum tolerable latency for each task is $T_n = 4$ s. The maximum computing capacity of a user device is 1 GHz, while the satellite provides a maximum computing capacity of 5 GHz [32]. The computation energy consumption coefficient is set to $k_n = 10^{-27}$, according to DVFS model [42]. For satellite–user links, the carrier frequency is 31 GHz [43], and each user is allocated a bandwidth of 1 MHz. The maximum transmit power of each user is 23 dBm [44], and the power density of AWGN is −174 dBM/Hz [45]. For inter-satellite links, the maximum data transmission rate is set to 40 Mbits/s. The important simulation parameters are shown in Table 2.

Table 2
Simulation Parameters.

Parameters	Value
Carrier communication frequency	31 GHz [43]
Communication bandwidth per user	1 MHz
User's maximum transmit power	23 dBm [44]
Power density of AWGN	−174 dBm/Hz [45]
User's maximum local computing capacity	1 GHz [32]
LEO's maximum computing capacity	5 GHz [32]
ISL maximum data transmission rate	40 Mbps
Required CPU cycles of one bit task	1000 cycles/bit [40,41]
Height of LEO satellites	1200 km [46]
Energy consumption coefficient	10^{-27} [42]

To validate the effectiveness of the proposed algorithms, this paper establishes the following three comparative schemes:

- *Local processing*: All users process their tasks locally without offloading task data to the LEO satellites.
- *Random offloading*: All users randomly select the target nodes for offloading their tasks, and the path selection strategy is based on the shortest path algorithm. The task splitting and resource allocation optimization follow the same algorithms as solving problems $\boldsymbol{P}1.1$ and $\boldsymbol{P}2.1$ in this paper.
- *NO-ISL*: All user tasks are offloaded to the respective satellites they are connected to for processing. There is no task offloading between satellites, meaning $\boldsymbol{\eta} = \boldsymbol{\beta}$, and $\boldsymbol{\zeta} = \mathbf{0}$. The task splitting and resource allocation optimization follow the same algorithms as solving problems $\boldsymbol{P}1.1$ and $\boldsymbol{P}2.1$ in this paper.

6.2. Convergence of proposed algorithm

First, this paper analyzes the convergence of the proposed algorithm under the two problems of delay minimization and energy consumption minimization, as shown in Figs. 5 and 6

Fig. 5 illustrates the convergence of the algorithm proposed in this paper for the delay minimization problem. The two curves represent the convergence of the algorithm with $N = 20$ and $N = 40$, respectively. From the variations observed in the two curves, it is evident that the proposed algorithm converges rapidly, reaching a relatively stable value within less than 5 iterations. This verifies the convergence of the algorithm proposed in this paper when solving the delay minimization problem. This result also helps us set the number of iterations in subsequent simulations. In subsequent simulations, we set the number of iterations for the simulation experiment of delay minimization problem to 7, which is slightly larger than 5.

Fig. 6 shows the convergence of proposed path scheduling and resource allocation algorithm for energy consumption minimization problem, where $N = 20$ and $N = 40$. When $N = 20$, the algorithm converges after the third iteration, and when $N = 40$, it converges after the fourth iteration. This validates the convergence of the algorithm proposed in this paper for energy consumption minimization. In subsequent simulations for the energy consumption minimization problem, the iteration count is set to 7.

6.3. Performance evaluation for delay minimization problem

As shown in Fig. 7, we evaluate the total delay against the different number of users. Specifically, we set the number of users from 10 to 50

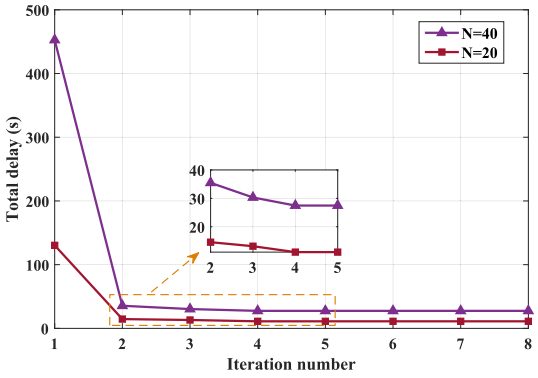


Fig. 5. Convergence of the proposed path scheduling and resource allocation algorithm for delay minimization.

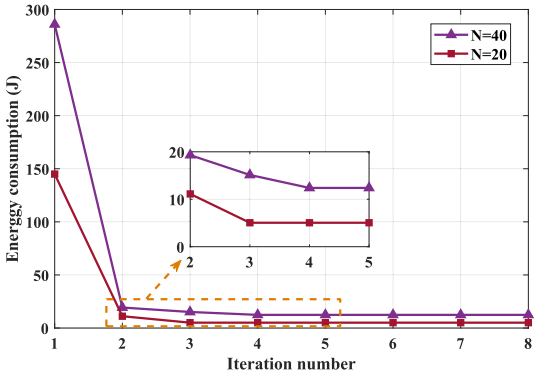


Fig. 6. Convergence of the proposed path scheduling and resource allocation algorithm for energy consumption minimization.

and evaluate the total system delay for four different path scheduling and resource allocation schemes. Comparing these four schemes across different user numbers, the proposed algorithm consistently achieves lower total delay compared to other three benchmark schemes, this validates the effectiveness of the algorithm proposed in this paper. With the increase in the number of users, the total latency of the four schemes shows an upward trend. This is partly due to the diminishing satellite resources available to each user as the number of users grows. Additionally, despite the individual latency for each user remaining unchanged, the total latency increases as the number of users increases. Comparing these four schemes, the total delays of the random strategy and the NO-ISL strategy are close. This is because although these two schemes use the computing resources of the satellite to help terrestrial users perform task processing, they fail to fully harness the computing and communication resources of the satellite network.

As shown in Fig. 8, we evaluate the system delay against the different input data size of tasks. Clearly, as the input task sizes increases, the overall task processing delay under the four schemes exhibits an upward trend. Notably, across different data sizes, the algorithm proposed in this paper consistently achieves the lowest system delay, highlighting the effectiveness of the proposed algorithm.

Fig. 9 illustrates the variation of the total processing delay under different numbers of serving satellites. As shown in the figure, with the increase in the number of serving satellites, the total processing delay decreases for all algorithms except the Local Processing algorithm. This is because more tasks can be offloaded to the satellite edge servers as the number of serving satellites increases, thereby reducing the computation burden on user devices. In addition, compared with the baseline algorithms, the proposed algorithm consistently achieves a lower processing delay under different numbers of serving satellites.

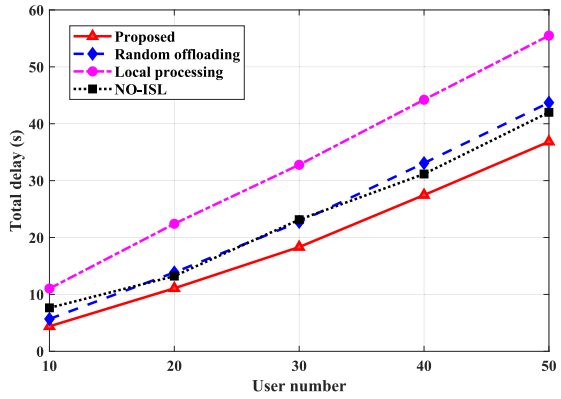


Fig. 7. Relationship between task processing delay and user number for delay minimization.

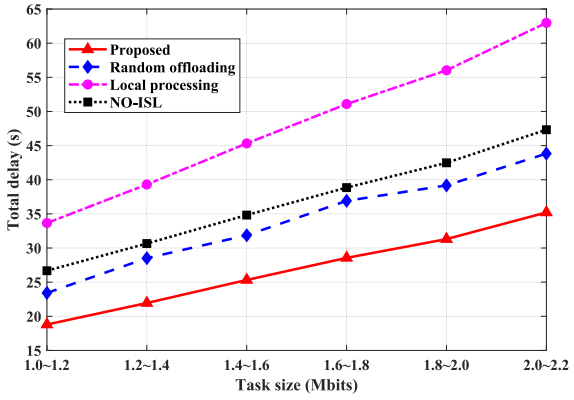


Fig. 8. Relationship between task processing delay and input task sizes.

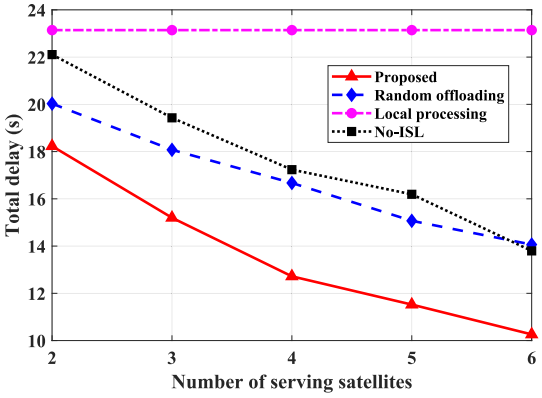


Fig. 9. Relationship between task processing delay and the number of serving satellites.

Fig. 10 illustrates the change in system delay as the local computation capability of user enhances, ranging from $1.0\times10^9 CPUcycles/s$ to $1.5\times10^9 CPUcycles/s$. With the increased local computation capacity, a larger data size of user can be processed locally, resulting in decreased overall system delay. In comparison to the other three baseline schemes, the algorithm proposed in this paper consistently achieves the minimum system latency across various local computation capabilities.

6.4. Performance evaluation for energy consumption minimization problem

Fig. 11 shows the variation in total user energy consumption under different user number. We set the user number from 10 to 50. With

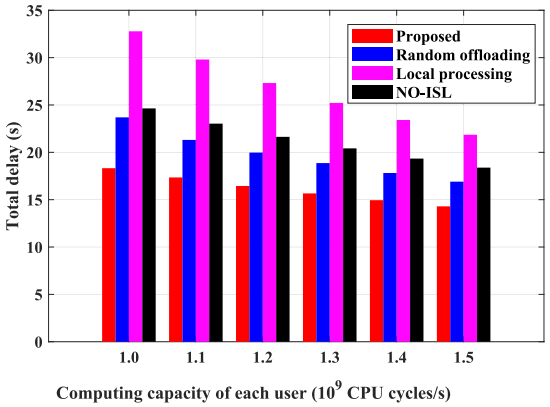


Fig. 10. Total delay versus the local computation capability.

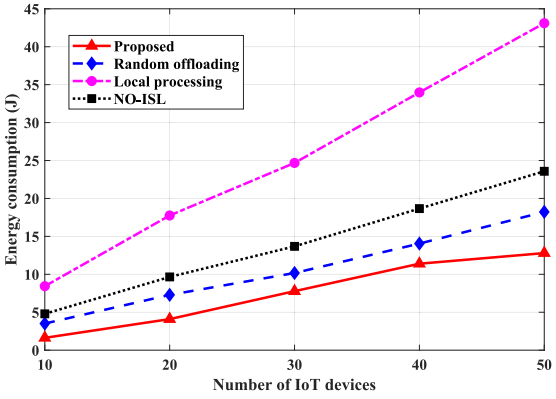


Fig. 11. Energy consumption versus the number of users.

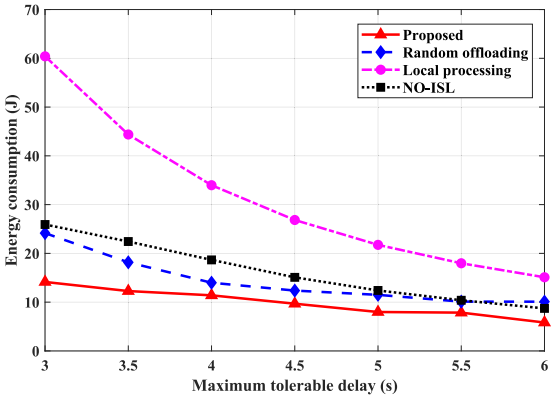


Fig. 12. Energy consumption versus the maximum tolerable delay.

the increase in the number of users, the energy consumption of all four schemes increases, and the proposed algorithm consistently exhibits the lowest energy consumption, which validates the effectiveness of the proposed algorithm in this paper.

Fig. 12 shows the users' energy consumption versus the maximum tolerable delay, where T = [3, 3.5, 4, 4.5, 5, 5.5, 6]. From Fig. 12, we can observe that as the maximum tolerable latency increases, users' energy consumption continues to decrease. Moreover, compared to the other three baseline schemes, the proposed algorithm in this paper consistently achieves the lowest users' energy consumption. In comparison to the *NO-ISL* scheme, the performance of the proposed algorithm shows a significant improvement, averaging around 39.14%.

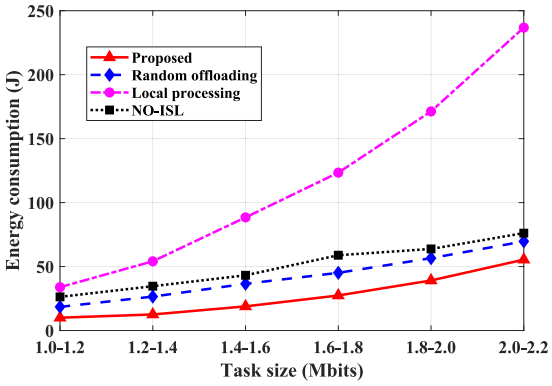


Fig. 13. Energy consumption versus the input task size.

We evaluate the energy consumption of different algorithms under varying task sizes, as shown in Fig. 13. From the figure, it can be observed that the total energy consumption increases with the growth of task data volume for all algorithms. Moreover, compared with the baseline algorithms, the proposed algorithm consistently achieves lower energy consumption, which demonstrates its effectiveness.

7. Conclusion

In this paper, we investigated the path scheduling and multiple resources allocation problem in LEO satellite-enhanced mobile edge computing network, taking into account both satellite–terrestrial and inter-satellite computation and communication resources. We formulate the minimization problem of the user utility function, and transformed it into two subproblems, including minimizing the system delay and minimizing the total user energy consumption. We then designed penalty-based successive convex approximation and alternating iteration algorithms to optimize and solve these problems, respectively. Additionally, we devised an initialization algorithm for path scheduling based on load balancing and the Floyd shortest path algorithm to obtain better initial solutions for the algorithm. Moreover, the results indicate that enabling inter-satellite multi-hop offloading effectively expands the available computing capacity and mitigates overload situations on individual satellites. Under the delay-oriented optimization mode, the proposed scheme significantly accelerates task completion, while under the energy-oriented mode, it effectively reduces the total user energy consumption. Simulation results show that the proposed algorithms can get smallest system delay and energy consumption, which verified the convergence and effectiveness of the proposed algorithm.

CRediT authorship contribution statement

Cheng Gao: Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Conceptualization. **Bo Hu:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Meng Meng:** Writing – original draft, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (NSFC) under Grant 61931005.

Data availability

No data was used for the research described in the article.

References

[1] C.-X. Wang, X. You, X. Gao, X. Zhu, Z. Li, C. Zhang, H. Wang, Y. Huang, Y. Chen, H. Haas, et al., On the road to 6G: Visions, requirements, key technologies, and testbeds, IEEE Commun. Surv. Tutor. 25 (2) (2023) 905–974.

[2] J. Radtke, C. Kebschull, E. Stoll, Interactions of the space debris environment with mega constellations—Using the example of the OneWeb constellation, Acta Astronaut. 131 (2017) 55–68.

[3] C.E. Fossa, R.A. Raines, G.H. Gunsch, M.A. Temple, An overview of the IRIDIUM (R) low earth orbit (LEO) satellite system, in: Proceedings of the IEEE 1998 National Aerospace and Electronics Conference. NAECON 1998. Celebrating 50 Years (Cat. No. 98CH36185), IEEE, 1998, pp. 152–159.

[4] T. Duan, V. Dinavahi, Starlink space network-enhanced cyber–physical power system, IEEE Trans. Smart Grid 12 (4) (2021) 3673–3675.

[5] S. Chen, S. Sun, S. Kang, System integration of terrestrial mobile communication and satellite communication—the trends, challenges and key technologies in B5G and 6G, China Commun. 17 (12) (2020) 156–171.

[6] A. Raman, M. Varvello, H. Chang, N. Sastry, Y. Zaki, Dissecting the performance of satellite network operators, Proc. ACM Netw. 1 (CoNEXT3) (2023) 1–25.

[7] D. Tuzi, E.F. Aguilar, T. Delamotte, G. Karabulut-Kurt, A. Knopp, Distributed approach to satellite direct-to-cell connectivity in 6G non-terrestrial networks, IEEE Wirel. Commun. 30 (6) (2023) 28–34.

[8] J.A. Fraire, O. Iova, F. Valois, Space-terrestrial integrated Internet of Things: challenges and opportunities, IEEE Commun. Mag. 60 (12) (2022) 64–70.

[9] A. Pastukh, V. Tikhvinskiy, E. Devyatkin, Exploring interference issues in the case of n25 band implementation for 5G/LTE direct-to-device NTN services, Sensors 24 (4) (2024) 1297.

[10] Q. Chen, W. Meng, T.Q. Quek, S. Chen, Multi-tier hybrid offloading for computation-aware IoT applications in civil aircraft-augmented SAGIN, IEEE J. Sel. Areas Commun. 41 (2) (2022) 399–417.

[11] X. Cao, B. Yang, Y. Shen, C. Yuen, Y. Zhang, Z. Han, H.V. Poor, L. Hanzo, Edge-assisted multi-layer offloading optimization of LEO satellite-terrestrial integrated networks, IEEE J. Sel. Areas Commun. 41 (2) (2022) 381–398.

[12] Y. Li, M. Wang, K. Hwang, Z. Li, T. Ji, LEO satellite constellation for global-scale remote sensing with on-orbit cloud AI computing, IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. (2023).

[13] Z. Ji, S. Wu, C. Jiang, Cooperative multi-agent deep reinforcement learning for computation offloading in digital twin satellite edge networks, IEEE J. Sel. Areas Commun. (2023).

[14] J. Liu, Y. Shi, Z.M. Fadlullah, N. Kato, Space-air-ground integrated network: A survey, IEEE Commun. Surv. Tutor. 20 (4) (2018) 2714–2741.

[15] X. Jia, T. Lv, F. He, H. Huang, Collaborative data downloading by using inter-satellite links in LEO satellite networks, IEEE Trans. Wirel. Commun. 16 (3) (2017) 1523–1532.

[16] M.M. Gost, I. Leyva-Mayorga, A. Pérez-Neira, M.Á. Vázquez, B. Soret, M. Moretti, Edge computing and communication for energy-efficient earth surveillance with LEO satellites, in: 2022 IEEE International Conference on Communications Workshops, ICC Workshops, IEEE, 2022, pp. 556–561.

[17] Y. Huang, X. Jiang, S. Chen, F. Yang, J. Yang, Pheromone incentivized intelligent multipath traffic scheduling approach for LEO satellite networks, IEEE Trans. Wirel. Commun. 21 (8) (2022) 5889–5902.

[18] Y. Lu, Y. Zhao, F. Sun, F. Yang, R. Liang, J. Shen, Z. Zuo, Enhancing transmission efficiency of mega-constellation LEO satellite networks, IEEE Trans. Veh. Technol. 71 (12) (2022) 13210–13225.

[19] Q. Tang, Z. Fei, B. Li, Z. Han, Computation offloading in LEO satellite networks with hybrid cloud and edge computing, IEEE Internet Things J. 8 (11) (2021) 9164–9176.

[20] X. Zhu, C. Jiang, Delay optimization for cooperative multi-tier computing in integrated satellite-terrestrial networks, IEEE J. Sel. Areas Commun. 41 (2) (2022) 366–380.

[21] F. Chai, Q. Zhang, H. Yao, X. Xin, R. Gao, M. Guizani, Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite IoT, IEEE Trans. Veh. Technol. 72 (6) (2023) 7783–7795.

[22] Z. Lin, M. Lin, B. Champagne, W.-P. Zhu, N. Al-Dhahir, Secure and energy efficient transmission for RSMA-based cognitive satellite-terrestrial networks, IEEE Wirel. Commun. Lett. 10 (2) (2020) 251–255.

[23] A. Li, T. Zhou, T. Xu, Y. Ouyang, H. Hu, C. Wu, LEO satellite assisted edge computing with latency and energy optimization, IEEE Trans. Netw. Sci. Eng. 12 (4) (2025) 2640–2653.

[24] P. Li, Y. Wang, Z. Wang, T. Wang, J. Cheng, Joint task offloading and resource allocation strategy for hybrid MEC-enabled LEO satellite networks: A hierarchical game approach, IEEE Trans. Commun. 73 (5) (2025) 3150–3166.

[25] W. Fan, Q. Meng, G. Wang, H. Bian, Y. Liu, Y. Liu, Satellite edge intelligence: DRL-based resource management for task inference in LEO-based satellite-ground collaborative networks, IEEE. Trans. Mob. Comput. 24 (10) (2025) 10710–10728.

[26] H. Li, J. Yu, L. Cao, Q. Zhang, Z. Song, S. Hou, Multi-agent reinforcement learning based computation offloading and resource allocation for LEO satellite edge computing networks, Comput. Commun. 222 (2024) 268–276.

[27] H. Zhang, H. Zhao, R. Liu, A. Kaushik, X. Gao, S. Xu, Collaborative task offloading optimization for satellite mobile edge computing using multi-agent deep reinforcement learning, IEEE Trans. Veh. Technol. 73 (10) (2024) 15483–15498.

[28] A.B. Wondmagegn, D. Won, D. Lee, J. Kim, J. Kim, S. Cho, Digital twin-driven semantic offloading for LEO-MEC-enabled remote IoT networks, Internet Things 35 (2026) 2101831.

[29] D.-J. Han, S. Hosseinalipour, D.J. Love, M. Chiang, C.G. Brinton, Cooperative federated learning over ground-to-satellite integrated networks: Joint local computation and data offloading, IEEE J. Sel. Areas Commun. 42 (5) (2024) 1080–1096.

[30] K. Wang, Y. Zhou, J. Li, L. Shi, W. Chen, L. Hanzo, Energy-efficient task offloading in massive MIMO-aided multi-pair fog-computing networks, IEEE Trans. Commun. 69 (4) (2020) 2123–2137.

[31] B. Di, H. Zhang, L. Song, Y. Li, G.Y. Li, Ultra-dense LEO: Integrating terrestrial-satellite networks into 5G and beyond for data offloading, IEEE Trans. Wirel. Commun. 18 (1) (2018) 47–62.

[32] C. Ding, J.-B. Wang, H. Zhang, M. Lin, G.Y. Li, Joint optimization of transmission and computation resources for satellite and high altitude platform assisted edge computing, IEEE Trans. Wirel. Commun. 21 (2) (2021) 1362–1377.

[33] Study on New Radio (NR) to Support Non-Terrestrial Networks (Release 15), Standard 3GPP TR 38.811 (V15.4.0).

[34] S. Ghosh, M.S. Rao, S. Singh, STARFIRE: An algorithm for estimating radio frequency interference in orbits around earth, Astron. Comput. 44 (2023) 100727.

[35] D. Wang, Y. Peng, X. Ma, W. Ding, H. Jiang, F. Chen, J. Liu, Adaptive wireless video streaming based on edge computing: Opportunities and approaches, IEEE Trans. Serv. Comput. 12 (5) (2018) 685–697.

[36] M.S. Hughes, B.J. Lunday, J.D. Weir, K.M. Hopkinson, The multiple shortest path problem with path deconfliction, EJOR 292 (3) (2021) 818–829.

[37] J. Chen, H. Xing, Z. Xiao, L. Xu, T. Tao, A DRL agent for jointly optimizing computation offloading and resource allocation in MEC, IEEE Internet Things J. 8 (24) (2021) 17508–17524.

[38] G. Michael, B. Stephen, CVX: Matlab software for disciplined convex programming, version 2.0 beta, 2013.

[39] J. Wang, Y. Sun, Z. Liu, P. Yang, T. Lin, Route planning based on floyd algorithm for intelligence transportation system, in: 2007 IEEE International Conference on Integration Technology, IEEE, 2007, pp. 544–546.

[40] D. Li, S. Wu, J. Jiao, N. Zhang, Q. Zhang, Age-aware task scheduling scheme in hybrid GEO-LEO satellite networks, in: GLOBECOM 2022-2022 IEEE Global Communications Conference, IEEE, 2022, pp. 5117–5122.

[41] Y. Wang, J. Yang, X. Guo, Z. Qu, Satellite edge computing for the internet of things in aerospace, Sensors 19 (20) (2019) 4375.

[42] H. Jiang, X. Dai, Z. Xiao, A. Iyengar, Joint task offloading and resource allocation for energy-constrained mobile edge computing, IEEE Trans. Mob. Comput. 22 (7) (2022) 4000–4015.

[43] C. Ding, J.-B. Wang, M. Cheng, M. Lin, J. Cheng, Dynamic transmission and computation resource optimization for dense LEO satellite assisted mobile-edge computing, IEEE Trans. Commun. 71 (5) (2023) 3087–3102.

[44] L. Chen, B. Hu, S. Chen, J. Xu, G. Xu, QoE-aware resource allocation for non-orthogonal multiple access enhanced HetNets, IEEE Trans. Cogn. Commun. Netw. 7 (4) (2021) 1363–1381.

[45] U. Mohammad, S. Sorour, M. Hefeida, Dynamic task allocation for mobile edge learning, IEEE Trans. Mob. Comput. 22 (12) (2021) 6860–6873.

[46] Y. Henri, The OneWeb satellite system, in: Handbook of Small Satellites: Technology, Design, Manufacture, Applications, Economics and Regulation, Springer, 2020, pp. 1091–1100.